

```
# Verify it works
numbers = [3, 7, 4, 9, 1, 6]
bubble_sort(numbers)
print(numbers) # Should print [1, 3, 4, 6, 7, 9]
```

Tired of different versions of the code for the same problem? How about asking the assistant to optimise the code instead? I just typed the following on the prompt:

```
optimize
```

The following is the result.

There are several ways you can optimize the bubble sort algorithm to make it run faster. Here are a few ideas:

Use a flag to keep track of whether any swaps were made on the last iteration of the loop. If no swaps were made, it means the list is already sorted and we can exit the loop early.

Only iterate up to the last element that has already been sorted. On each pass, the largest element will "bubble" up to the end of the list, so there's no need to compare it to any other elements on the next pass.

Use a different sorting algorithm altogether, such as quick sort or merge sort, which have better average-case time complexity than bubble sort.

I hope these suggestions are helpful! Let me know if you have any questions.

I am sure you noticed that the assistant is remembering the whole conversation and answering the questions contextually.

Development with ChatGPT

The above conversation only illustrates how the assistant can help us in writing code for smaller problems. However, developing software is not just about writing code. Let's act as if we do not know much about developing Spring Boot applications and see how much the assistant helps us.

I started by asking the following question:

```
difference between log4j and slf4j
```

It gave a whole lot of information. Try this yourself and find out how useful it is.

Then, I asked the following question:

```
spring program with slf4j
```

It generated the following code:

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class MySpringApp {
    private static final Logger logger = LoggerFactory.getLogger(MySpringApp.class);
```

```
public static void main(String[] args) {
    SpringApplication.run(MySpringApp.class, args);
    logger.info("Spring application started!");
}
```

The assistant does not wash its hands off with just code. It gives a lot of notes as well. For instance, the following is the note generated along with the above code.

In this example, we use the LoggerFactory class from slf4j to create a logger for the MySpringApp class. We can then use the logger object to output log statements at different levels (e.g., info, debug, error).

To use slf4j in a Spring application, you will need to include the slf4j-api library as a dependency in your project. You will also need to include a concrete implementation of slf4j, such as log4j, logback, or java.util.logging.

For example, to use log4j as the underlying logging implementation, you will need to include the log4j library as a dependency in your project. You can then configure log4j to output log statements to the desired destination (e.g., console, file).

Since it is advising some dependencies, I acted as if I do not know how it can be done. So, I asked this follow-up question:

```
pom file for spring with slf4j
```

Here is the resulting suggestion:

```
<project>
  <!-- Other project configuration -->

  <dependencies>
    <!-- Other dependencies -->
    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-api</artifactId>
      <version>1.7.30</version>
    </dependency>
    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-log4j12</artifactId>
      <version>1.7.30</version>
    </dependency>
  </dependencies>
</project>
```

Again, the notes suggested a log4j dependency. I asked the assistant to add that too, and it generated the following refined configuration.

```
<project>
  <!-- Other project configuration -->

  <dependencies>
    <!-- Other dependencies -->
```